

Tarea 2 — Migración a una arquitectura asíncrona con Apache Kafka

Plataforma distribuida de consultas geoespaciales con caché

Gabriel González - Maximiliano Solorza

3 de junio de 2026

Resumen

Este informe documenta la evolución del sistema distribuido de consultas geoespaciales de la Tarea 1 (cadena síncrona HTTP) hacia una **arquitectura asíncrona dirigida por eventos** basada en **Apache Kafka** (modo KRaft). Se incorpora un grupo de consumidores escalable, una política de **tolerancia a fallos** con reintentos y cola de mensajes muertos (*Dead Letter Queue*, DLQ), y una expansión del sistema de métricas para monitorear la **salud de las colas**. La ruta síncrona original se preserva como modo seleccionable (*Sistema Base*). Se valida el sistema mediante una batería de cinco escenarios que cubren la Definición de Terminado (DoD). Los resultados muestran que el desacoplamiento mediante colas convierte la latencia de bloqueo en *backlog* elástico (*recovery rate* = 0,90 ante fallas, absorción de un *spike* 40 → 250 q/s sin pérdidas), y que el escalamiento de consumidores mejora la latencia y la recuperación, aunque el *throughput* sostenido queda acotado por el cuello de botella del backend.

Índice

1. Introducción y objetivos	2
2. Arquitectura del sistema	2
2.1. Visión general	2
2.2. Servicios	3
2.3. Estrategia de tópicos	3
2.4. Esquema del mensaje (enriquecido)	3
2.5. Decisiones técnicas y justificación	3
3. Análisis de cumplimiento	4
4. Diseño e implementación	5
4.1. Productor asíncrono y <i>spike</i>	5
4.2. Grupo de consumidores y procesamiento	5
4.3. Política de reintentos, <i>backoff</i> y DLQ	5
4.4. Métricas de salud de la cola	6
5. Metodología experimental	6
6. Resultados y análisis	6
6.1. Throughput y desacoplamiento (DoD 1 y 2)	7
6.2. Latencia extremo a extremo (E2E)	7
6.3. Escalamiento de consumidores (DoD 3)	8
6.4. Tolerancia a fallos y recuperación (DoD 4)	9
6.5. Absorción del <i>spike</i> (DoD 5)	9
7. Discusión	10
8. Conclusiones	10
A. Reproducción	10
B. Parámetros clave (.env)	11

1. Introducción y objetivos

La Tarea 1 implementó un sistema de caché distribuido sobre Redis para consultas geoespaciales (Q1–Q5) acerca de las edificaciones de cinco zonas de Santiago. Su arquitectura era **síncrona**: el Generador de Tráfico invocaba por **HTTP** al Servicio de Caché, que en caso de *miss* delegaba de forma bloqueante al Generador de Respuestas. Este acoplamiento fuerte implica que un pico de tráfico o una caída del backend se propagan inmediatamente al cliente.

La Tarea 2 desacopla la generación del procesamiento mediante un **bus de mensajería Kafka**. Los objetivos son:

1. Introducir Kafka y un esquema de tópicos que aisle el tráfico normal, los reintentos y los descartes.
2. Enriquecer el mensaje con metadatos de trazabilidad (UUID, contador de reintentos, *timestamp*).
3. Construir un grupo de consumidores concurrente y **escalable**.
4. Tolerar caídas del Generador de Respuestas sin perder consultas (reintentos → DLQ).
5. Monitorear el comportamiento asíncrono: *throughput*, latencias, *retry/recovery/DLQ rate*, *backlog* y *recovery time*.

2. Arquitectura del sistema

2.1. Visión general

La Figura conceptual del flujo asíncrono es la siguiente:

```
traffic_generator (mode=kafka)
  | produce {uuid, query_type, params, retry_count, created_at}
  v
[ consultas_principales ] (6 particiones)
  |
  consumer-group "cache-consumers" (N réplicas escalables)
    | POST cache_service/query
    +-----+-----+
200 OK (HIT/MISS) 502 (Generador de Respuestas caído)
  | retry_count += 1 ; backoff
"processed" < MAX_RETRIES -> [ consultas_reintentos ]
                        >= MAX_RETRIES -> [ consultas_dlq ] (terminal)
                        |
                        (reintentos reconsumidos por el mismo grupo; éxito -> "recovered")

metrics_service: consume eventos + sondea el lag del grupo (AdminClient)
                  -> Backlog size / Recovery time
```

Listing 1: Flujo de procesamiento asíncrono

2.2. Servicios

Servicio	Puerto	Rol
kafka	9092	Broker KRaft (sin Zookeeper), nodo único. Hospeda los 3 tópicos.
traffic_generator	8000	Productor. Genera Q1–Q5 con distribuciones Zipf/Uniforme; modo <code>sync</code> o <code>kafka</code> .
consumer	8004*	Grupo <code>cache-consumers</code> . Puente Kafka↔HTTP; enruta fallas a reintentos/DLQ. Escalable.
cache_service	8001	Proxy de Redis (LRU/LFU/FIFO). Sin cambios respecto de Tarea 1.
response_generator	8002	Calcula Q1–Q5; endpoint <code>/admin/fail</code> para simular caídas.
metrics	8003	KPIs síncronos + <code>async_kpis</code> + <code>queue_health</code> .
redis	6379	Almacén del caché (fijo en LRU / 200 MB).

Cuadro 1: Servicios del sistema. *El puerto del `consumer` es interno (no se publica al host) para permitir réplicas.

2.3. Estrategia de tópicos

Se implementan **exactamente tres** tópicos, decisión de diseño que permite medir directamente los KPIs sin contadores mezclados:

- `consultas_principales` (6 particiones): tráfico normal. Las 6 particiones habilitan el balanceo de carga al escalar consumidores.
- `consultas_reintentos` (3 particiones): mensajes que fallaron y serán reprocesados. Aísla el *backlog* puro del de reintentos.
- `consultas_dlq` (1 partición): mensajes expulsados tras agotar los reintentos. Su *offset* da el *DLQ rate* directo.

2.4. Esquema del mensaje (enriquecido)

Cada mensaje publicado incluye obligatoriamente los campos de trazabilidad:

```
{ "uuid": "f47ac10b-58cc-4372-...", "query_type": "Q1",  
  "params": { "zone_id": "Z1", "confidence_min": 0.8 },  
  "retry_count": 0, "created_at": 1717459200.123 }
```

El `uuid` permite trazar un mensaje a través de los tres tópicos; el `retry_count` dirige la política de reintentos; el `created_at` permite calcular la latencia extremo a extremo (incluida la espera en cola).

2.5. Decisiones técnicas y justificación

- **Cliente Kafka:** `confluent-kafka`. Elegido por su alto rendimiento (basado en `librdkafka`) en escenarios de alta concurrencia, crítico para absorber el *spike*.
- **Caché congelada (LRU / 200 MB).** Validada en Tarea 1; el foco de esta entrega es la tolerancia a fallos y la salud de las colas, no re-evaluar políticas de evicción.
- **Reutilización de `cache_service`.** El `consumer` es un puente delgado que delega en el servicio de caché existente (`POST /query`); captura el HTTP 502 como señal de caída del backend para activar los reintentos. No se modificó la lógica de caché.

3. Análisis de cumplimiento

La Tabla 2 mapea cada tarea de las Épicas y cada escenario de la Definición de Terminado con su implementación y su evidencia (archivo de código y/o resultado experimental).

Cuadro 2: Matriz de cumplimiento de Épicas y DoD.

Ítem	Requisito	Implementación / Evidencia	Estado
<i>Épica 1 — Infraestructura de Mensajería</i>			
1.1	Broker Kafka desplegado	Servicio <code>kafka</code> (KRaft) en <code>docker-compose.yml</code>	Cumplido
1.2	Tópicos del sistema	3 tópicos creados idempotentemente vía <code>AdminClient</code> (<code>consumer/app/main.py:ensure_topics</code>)	Cumplido
<i>Épica 2 — Adaptación del Generador (Productor)</i>			
2.1	Publicar <code>consultas_principales</code>	a <code>Producer</code> de <code>confluent-kafka</code> (<code>_produce</code>); modo <code>kafka</code>	Cumplido
2.2	Payload con <code>UUID</code> , <code>retry_count=0</code> , <code>timestamp</code>	en <code>_enrich()</code> <code>traffic_generator/app/main.py</code>	Cumplido
2.3	Mantener Zipf y Uniforme	Se reutiliza <code>distributions.py</code> sin cambios	Cumplido
<i>Épica 3 — Grupo de Consumidores</i>			
3.1	Consumidor con <i>Consumer Group</i>	Grupo <code>cache-consumers</code> suscrito a <code>consultas_principales</code>	Cumplido
3.2	Interacción caché/backend (Hit/Miss)	<code>_process()</code> : <code>cache_service/query</code> ; POST Hit→éxito, Miss→backend	Cumplido
3.3	Soporte de escalamiento (réplicas)	<code>docker compose up -scale consumer=N</code> ; 6 particiones. Verificado : 2 miembros, particiones 3/3 (Fig. 3)	Cumplido
<i>Épica 4 — Tolerancia a Fallos</i>			
4.1	Simulación de fallas	POST <code>/admin/fail</code> y parada manual del contenedor	Cumplido
4.2	Política de reintentos (+1, → reintentos)	<code>_process()</code> captura el 502, incrementa <code>retry_count</code> , re-encola con <i>backoff</i>	Cumplido
4.3	Consumo de reintentos	El mismo grupo se suscribe a <code>consultas_reintentos</code>	Cumplido
4.4	Enrutamiento a DLQ	<code>retry_count ≥ MAX_RETRIES</code> → <code>consultas_dlq</code>	Cumplido
<i>Épica 5 — Expansión de Métricas</i>			
5.1	<i>Throughput</i> , p50/p95, <i>retry/recovery</i> /DLQ rate	<code>metrics.summary().async_kpis</code> (Tabla 4)	Cumplido
5.2	<i>Backlog size</i> y <i>Recovery time</i>	<code>KafkaMonitor</code> (lag del grupo) → <code>queue_health</code>	Cumplido
<i>Definición de Terminado (escenarios)</i>			
DoD 1	Sistema Base síncrono	<code>mode=sync</code> ; <code>snap_base_sync.json</code>	Cumplido

Ítem	Requisito	Implementación / Evidencia	Estado
DoD 2	Kafka + 1 consumidor	<code>snap_kafka_1c.json</code>	Cumplido
DoD 3	Kafka + múltiples consumidores	<code>snap_kafka_3c.json</code>	Cumplido
DoD 4	Falla temporal + recuperación	<code>snap_failure.json</code> (<i>recovery rate</i> = 0,90)	Cumplido
DoD 5	Spike de tráfico	<code>snap_spike.json</code> (pico 40 → 250 q/s absorbido)	Cumplido

Resumen de cumplimiento

Las **5 Épicas** (13 tareas) y los **5 escenarios** de la Definición de Terminado se encuentran implementados y verificados experimentalmente. No hay ítems pendientes.

4. Diseño e implementación

4.1. Productor asíncrono y *spike*

En modo `kafka`, el Generador de Tráfico mantiene su lógica de llegadas de Poisson y los selectores Zipf/Uniforme, pero en lugar de esperar la respuesta HTTP publica el mensaje enriquecido y continúa. La operación `produce()` es no bloqueante (buffer local con *flush* en hilo de fondo), lo que permite sostener tasas muy altas. El *spike* se modela conmutando la tasa de llegadas (`spike_at_sec`, `spike_rate_qps`) a mitad del experimento.

4.2. Grupo de consumidores y procesamiento

Cada réplica del `consumer` ejecuta un bucle de *poll* bloqueante en un hilo de fondo (modelo natural de `confluent-kafka`) y expone `/health` para el *healthcheck*. Todas las réplicas comparten el `group.id`, de modo que Kafka reparte automáticamente las particiones entre ellas (*rebalancing*). El *commit* de *offsets* es manual tras procesar cada mensaje (semántica *at-least-once*).

4.3. Política de reintentos, *backoff* y DLQ

Cuando `cache_service` responde 502 (porque el Generador de Respuestas está caído), el consumidor incrementa `retry_count` y:

- si `retry_count < MAX_RETRIES (=3)`: espera un *backoff* (`RETRY_DELAY_MS=1000 ms`) y re-publica a `consultas_reintentos`;
- si `retry_count ≥ MAX_RETRIES`: publica a `consultas_dlq`.

El *backoff* es la decisión de diseño que hace que los reintentos *esperen* a la recuperación del backend en lugar de agotarse instantáneamente hacia la DLQ; sin él, una caída de pocos segundos enviaría todo a la DLQ y la política de reintentos sería inútil.

4.4. Métricas de salud de la cola

El servicio de métricas ejecuta un `KafkaMonitor` que, con un `Consumer` en el mismo grupo (sin suscribirse), lee los *offsets* comprometidos y los *high-watermarks* por partición para calcular el *lag* (backlog) de cada tópico. El *recovery time* se mide como el tiempo desde el pico de backlog hasta que la cola se vacía.

5. Metodología experimental

Se ejecutó la batería `experiments/run_async.py -scenario all` sobre el mismo *stack* (caché LRU/200 MB fija). La Tabla 3 resume la configuración de cada escenario.

Escenario	Config.	Objetivo (DoD)
<code>base_sync</code>	sync, 60 q/s, 40 s	Sistema Base sin Kafka.
<code>kafka_1c</code>	kafka, 60 q/s, 40 s, 1 cons.	Tránsito fluido con un worker.
<code>kafka_3c</code>	kafka, 60 q/s, 40 s, 3 cons.	Mejora por escalamiento.
<code>failure</code>	kafka, 60 q/s, 60 s, 2 cons.	Caída del backend (~18 s) y recuperación.
<code>spike</code>	kafka, 40 → 250 q/s @ 20 s, 2 cons.	Absorción de ráfaga.

Cuadro 3: Configuración de los cinco escenarios.

6. Resultados y análisis

Escenario	Hit rate	Thr. (q/s)	E2E p50 (s)	E2E p95 (s)	Retry rate	Recov. rate	DLQ rate	Backlog pico
<code>base_sync</code>	0,859	47,9	—	—	—	—	—	0
<code>kafka_1c</code>	0,881	55,6	5,13	8,00	0,000	—	0,000	461
<code>kafka_3c</code>	0,882	57,2	1,45	7,83	0,000	—	0,000	393
<code>failure</code>	0,907	55,7	14,57	25,31	0,011	0,903	0,001	1319
<code>spike</code>	0,945	106,6	16,20	17,51	0,000	—	0,000	2743

Cuadro 4: Resultados globales por escenario (E2E = latencia extremo a extremo, incluye espera en cola). Fuente: `results/snap_*.json`.

6.1. Throughput y desacoplamiento (DoD 1 y 2)

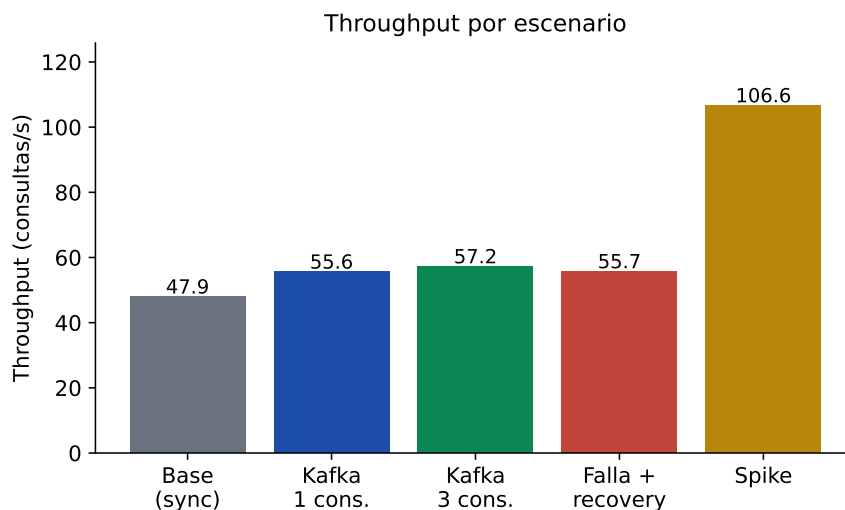


Figura 1: Throughput procesado por escenario.

El modo asíncrono (`kafka_1c`, 55,6 q/s) iguala o supera al `base_sync` (47,9 q/s) en *throughput*, pero el cambio cualitativo clave está en *dónde vive la latencia*. En modo síncrono, el cliente sufre una latencia de *miss* de hasta **4,27 s** en p95: 16 *workers* concurrentes golpean un Generador de Respuestas de un solo hilo con cómputo `pandas` bloqueante, generando contención. En modo

asíncrono, esa misma operación vista por `cache_service` cae a $\sim 0,27$ s (p95 de *miss*) porque los consumidores la *serializan*; la espera no desaparece, se traslada al **backlog de la cola** (Fig. 2). **Conclusión:** el desacoplamiento convierte la latencia de bloqueo del cliente en *backlog* elástico — el productor nunca se bloquea, lo que habilita la absorción de ráfagas y la tolerancia a fallos.

6.2. Latencia extremo a extremo (E2E)

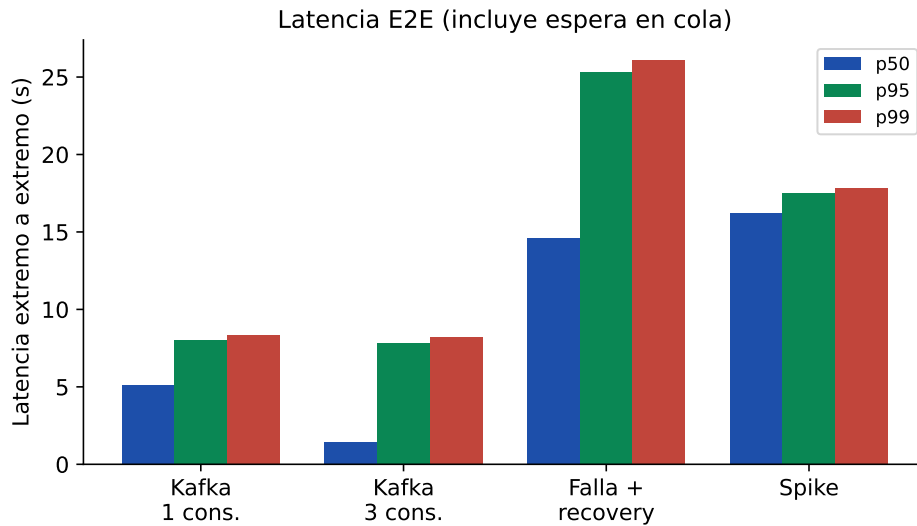


Figura 2: Latencia E2E (desde `created_at` hasta el procesamiento). Incluye el tiempo de espera en cola.

Las latencias E2E de varios segundos no indican un sistema lento, sino que el ritmo de *ingesta* (60–250 q/s) supera la capacidad del backend, por lo que los mensajes esperan en cola. Es el comportamiento esperado y deseable: la cola actúa como amortiguador. El caso **failure** exhibe la mayor p95 (25,3 s) porque, durante la caída, los *misses* se acumulan; el caso **spike** mantiene una p95 menor (17,5 s) pese al mayor volumen, porque opera con 2 consumidores y nunca pierde el backend.

6.3. Escalamiento de consumidores (DoD 3)

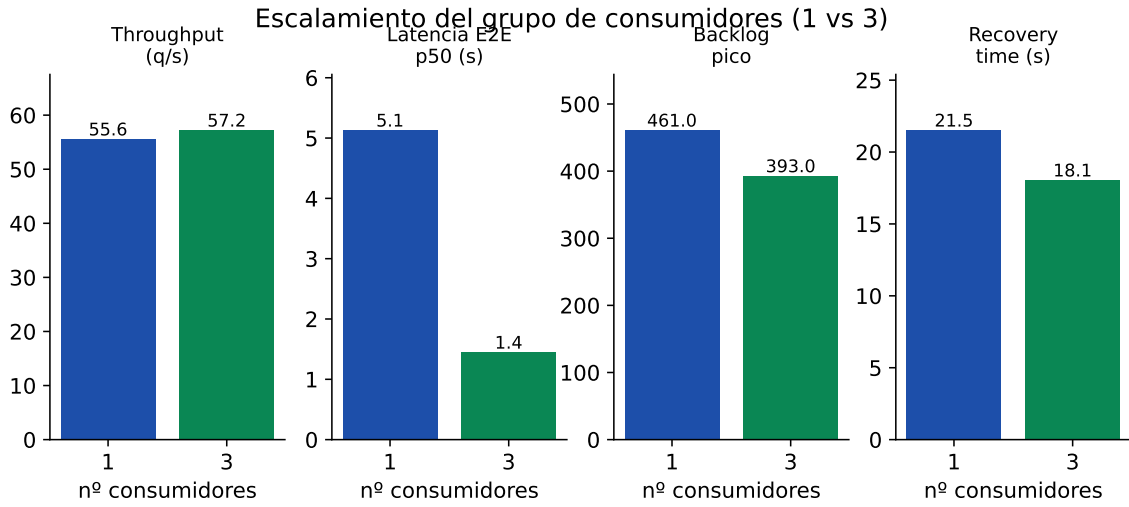


Figura 3: Escalamiento de 1 a 3 consumidores: throughput, latencia E2E p50, backlog pico y recovery time.

Pasar de 1 a 3 consumidores casi no mueve el *throughput* sostenido ($55,6 \rightarrow 57,2$ q/s). La razón es de manual de sistemas distribuidos: el **cuello de botella es el Generador de Respuestas** (un solo *worker* con cómputo bloqueante de 30–120 ms), no la capa de consumo. Sin embargo, escalar **sí** mejora otras dimensiones: la latencia E2E mediana cae **3,5×** ($5,13 \rightarrow 1,45$ s), el *backlog* pico baja ($461 \rightarrow 393$) y el *recovery time* mejora ($21,5 \rightarrow 18,1$ s). Es decir, más consumidores drenan los *hits* (que no tocan el backend) en paralelo y vacían la cola más rápido. **Lección:** escalar la capa de consumo ayuda hasta el siguiente cuello de botella; para aumentar el *throughput* sostenido habría que escalar también el Generador de Respuestas.

6.4. Tolerancia a fallos y recuperación (DoD 4)

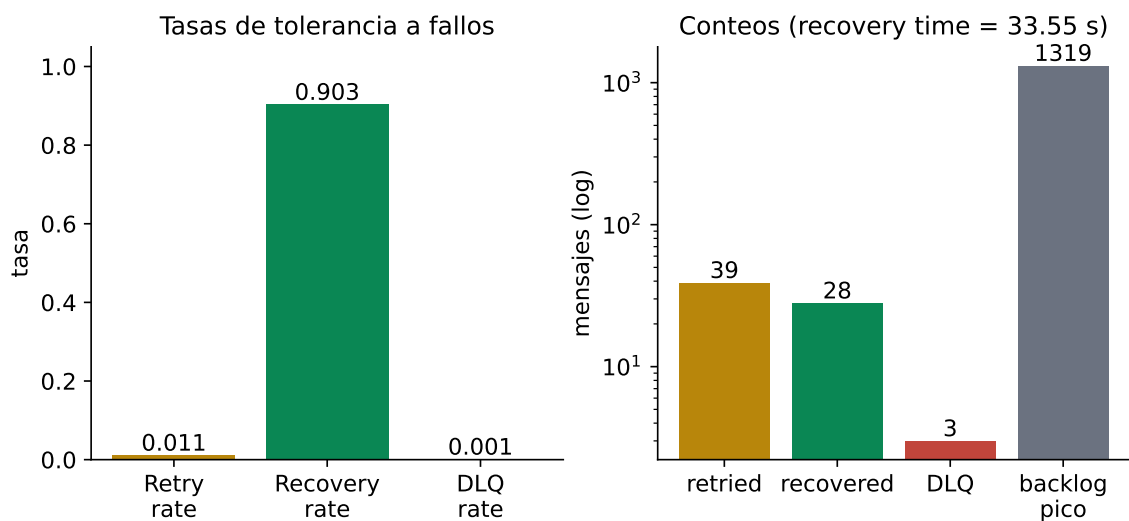


Figura 4: Escenario de falla: tasas y conteos de reintentos, recuperación y DLQ.

Durante la caída simulada del Generador de Respuestas se dispararon **39 reintentos**; de los mensajes que entraron al *pipeline* de reintentos, el **90,3 %** se recuperó (*recovery rate* = 0,903:

28 recuperados, solo 3 a la DLQ). El *backlog* creció hasta **1319** mensajes y se drenó por completo en **33,5 s** tras restaurar el backend (Fig. 5). Dos observaciones:

- La *hit rate* se mantuvo alta (0,907) durante la falla: una caída del Generador de Respuestas no afecta a los *hits* cacheados, que se siguen sirviendo. El sistema degrada parcialmente, no totalmente.
- El *backoff* de reintentos es lo que produce el alto *recovery rate*: los mensajes esperan a que el backend vuelva en lugar de agotarse hacia la DLQ. **Ninguna consulta se perdió en la red**; las 3 que llegaron a la DLQ quedan disponibles para inspección.

6.5. Absorción del *spike* (DoD 5)

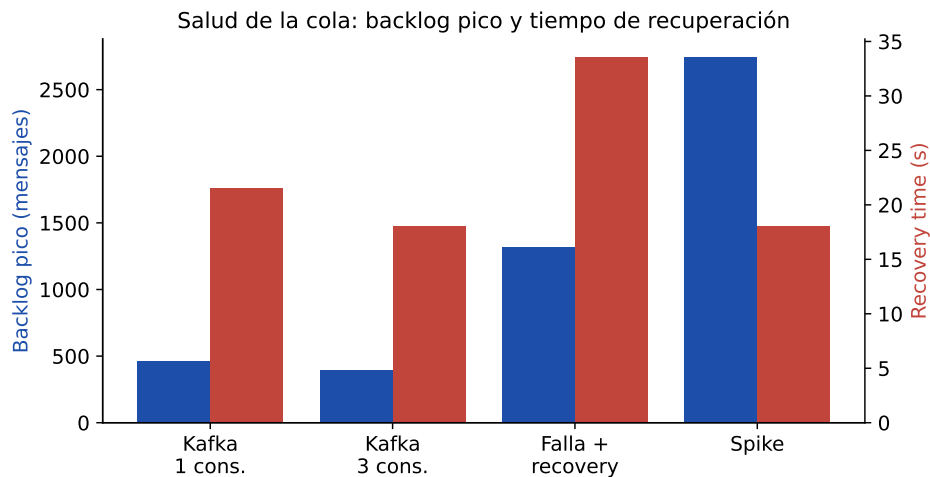


Figura 5: Salud de la cola: backlog pico y recovery time por escenario.

Al conmutar la tasa de 40 a 250 q/s, el sistema **no colapsó**: procesó 7564 mensajes (throughput promedio 106,6 q/s), el *backlog* absorbió el exceso (pico **2743**) y se drenó en **18,1 s**, con **0 errores y 0 mensajes en la DLQ**. Esto evidencia la propiedad central de las arquitecturas dirigidas por eventos: la cola desacopla la tasa de producción de la de consumo, suavizando ráfagas que en el modelo síncrono habrían saturado al backend y propagado errores al cliente.

7. Discusión

- **Síncrono vs. asíncrono.** El modo síncrono ofrece menor latencia por petición *cuando hay holgura*, pero acopla cliente y backend: bajo carga, la latencia de *miss* se dispara (4,27 s p95). El modo asíncrono nunca bloquea al productor y transforma la sobrecarga en *backlog* medible y recuperable.
- **Escalabilidad acotada por el backend.** El experimento confirma la teoría de colas: añadir consumidores reduce latencia y acelera la recuperación, pero el *throughput* de estado estable está limitado por el servicio más lento de la cadena.
- **Tolerancia a fallos efectiva.** La combinación de reintentos con *backoff* + DLQ logra recuperar el 90 % de los mensajes ante una caída transitoria, sin pérdidas, y aísla en la DLQ los casos irrecuperables.

Limitaciones. El broker Kafka es de nodo único (*replication factor* 1), adecuado para la evaluación pero no tolerante a la caída del propio broker. El *recovery time* se mide a nivel de

drenado de backlog (no por mensaje individual). El Generador de Respuestas, al ser monohilo y bloqueante, acota el *throughput* global.

8. Conclusiones

La migración a una arquitectura asíncrona basada en Kafka cumple la totalidad de las Épicas y de la Definición de Terminado. El sistema resultante: (i) desacopla ingesta y procesamiento, absorbiendo un *spike* de 40 → 250 q/s sin pérdidas; (ii) escala horizontalmente el grupo de consumidores, mejorando la latencia mediana $3,5\times$ y el *recovery time*; y (iii) tolera caídas del backend recuperando el 90 % de las consultas mediante reintentos con *backoff* y aislando el resto en una DLQ. Los KPIs de salud de cola (*backlog*, *recovery time*) y de tolerancia a fallos (*retry/-recovery/DLQ rate*) hacen observable el comportamiento asíncrono, y confirman empíricamente las propiedades esperadas de un sistema distribuido dirigido por eventos.

A. Reproducción

```
# Levantar el stack
docker compose up -d --build

# Bateria completa (5 escenarios del DoD)
python experiments/run_async.py --scenario all

# Figuras del informe
python experiments/build_figures_async.py

# Inspección del grupo / lag por tópico
docker compose exec kafka /opt/kafka/bin/kafka-consumer-groups.sh \
  --bootstrap-server localhost:9092 --describe --group cache-consumers
```

Listing 2: Comandos para reproducir la batería

B. Parámetros clave (.env)

```
MAX_RETRIES=3 # fallas antes de expulsar a la DLQ
RETRY_DELAY_MS=1000 # backoff antes de re-encolar a reintentos
PARTITIONS_MAIN=6 # particiones del topico principal (>= replicas)
CACHE_POLICY=LRU # cache congelada
REDIS_MAXMEMORY=200mb
```